Feuille de TD n° 1

Bases de la programmation – Les conditions

Rappels Python

Afficher. L'instruction `print` affiche ses arguments, séparés par des espaces :

```
A = 12
print("Bonjour")
print(A)
print("La valeur contenue dans la variable A est", A)
```

Ce qui s'affiche à l'écran :

```
Bonjour
12
La valeur contenue dans la variable A est 12
```

Lire une valeur au clavier. L'instruction `input` affiche un message, attend que l'utilisateur tape une valeur, puis renvoie cette valeur sous la forme d'une *chaîne de caractères*. Pour obtenir un nombre, il faut la convertir avec `int` (entier) ou `float` (réel) :

```
a = input("Entrez une valeur : ")        # a est une chaîne de caractères
n = int(input("Entrez un entier : "))    # n est un entier
x = float(input("Entrez un réel : "))    # x est un réel
```

Racine carrée. La fonction `sqrt` du module `math` calcule une racine carrée :

```
from math import sqrt
print(sqrt(16))                          # affiche 4.0
```

Exercice 1 Comparaison de deux entiers

Écrire un programme qui demande à l'utilisateur deux entiers, stockés dans deux variables `a` et `b`, et qui affiche « plus grand » si `a` est strictement supérieur à `b`. Dans le cas contraire, le programme affiche « plus petit ou égal ».

Modifier ensuite le programme afin qu'il affiche l'un des trois messages suivants : « plus grand », « plus petit » ou « égal ».

Exercice 2 Aire d'un rectangle

Écrire un programme qui demande à l'utilisateur une longueur et une largeur (deux nombres réels), stockées dans deux variables `longueur` et `largeur`. La longueur doit être supérieure ou égale à la largeur.

Si cette condition est respectée, le programme affiche l'aire du rectangle correspondant. Sinon, il affiche le message « Données incohérentes ».

Exercice 3 Contrôle de saisie

Écrire un programme qui lit six entiers et les stocke dans six variables : `a`, `b`, `c`, `d`, `e` et `f`.

Si au moins trois de ces variables sont non nulles, le programme affiche « La saisie est correcte ». Sinon, il affiche « La saisie est incorrecte ».

Exercice 4 Tri de trois nombres

Écrire un programme qui lit trois nombres et les stocke dans trois variables `a`, `b` et `c`. Le programme doit ensuite réordonner leur contenu de manière à obtenir :

$$a \leq b \leq c.$$

Afficher les trois variables à la fin du programme afin de vérifier le résultat.

Exercice 5 Minimum, maximum et moyenne

Écrire un programme qui permet de saisir dix valeurs réelles et qui affiche la plus petite valeur, la plus grande valeur et la moyenne des dix valeurs. Les boucles n'ayant pas encore été vues, on répétera dix fois le bloc de saisie. Le programme ne doit utiliser que **quatre variables** : la valeur saisie, le minimum, le maximum et la somme.

Exercice 6 Distance entre deux points

Écrire un programme qui saisit les coordonnées entières de deux points du plan et qui affiche la distance les séparant sous la forme d'un nombre réel (voir les rappels pour la racine carrée).

Exercice 7 Triangle rectangle

Écrire un programme qui saisit les coordonnées entières de trois points du plan et qui indique si ces trois points forment un triangle rectangle. On utilisera le théorème de Pythagore sur les carrés des longueurs des côtés (aucune racine carrée n'est nécessaire). Le programme affiche « Triangle rectangle » ou « Pas un triangle rectangle » ; si deux des points sont identiques, il affiche « Deux points sont confondus ».

Exercice 8 Calculatrice

Écrire un programme qui demande à l'utilisateur une opération parmi `+`, `-`, `*` et `/`, puis deux opérandes entiers. Le programme affiche le résultat de l'opération appliquée aux deux opérandes ; pour la division, il affiche le quotient entier (`//`). Si l'opération n'est pas l'une des quatre, le programme affiche « Opération inconnue ».

> **Attention :** pour la division, pensez à interdire la division par zéro.

Exemple d'exécution :

```
Entrez l'opération : *
Entrez le premier opérande : 3
Entrez le deuxième opérande : 4
Le résultat de 3 * 4 est 12
```

Exercice 9 Équation du second degré

Écrire un programme qui demande trois valeurs réelles `a`, `b` et `c`, puis affiche les solutions réelles de l'équation

$$ax^2 + bx + c = 0.$$

Si `a` est nul, le programme affiche « a doit être différent de zéro » et s'arrête. Sinon, selon le signe du discriminant $\Delta = b^2 - 4ac$, il affiche « Pas de solution réelle » ($\Delta < 0$), la solution double ($\Delta = 0$) ou les deux solutions ($\Delta > 0$).

Exercice 10 Chiffres d'un nombre

Écrire un programme qui reçoit un entier positif à quatre chiffres et qui affiche ses quatre chiffres, séparés par des espaces.

Le programme doit d'abord vérifier que le nombre contient bien quatre chiffres. Dans le cas contraire, il affiche : « Le nombre ne contient pas 4 chiffres ».

Exemple : pour 1234, le programme affiche `1 2 3 4`.

Feuille de TD n° 1 – Corrigé

Bases de la programmation – Les conditions

Remarque générale : les solutions proposées n'utilisent que les notions de la feuille : lecture au clavier avec `input`, conversions `int` / `float`, affichage avec `print`, instructions conditionnelles `if` / `elif` / `else` et `sqrt`. Pour l'exercice 5, une version avec boucle est donnée en complément, à revoir après le TD n° 2. D'autres solutions correctes sont bien sûr possibles.

Exercice 1 Comparaison de deux entiers

Version 1 : deux messages

```
a = int(input("Entrez un premier entier : "))
b = int(input("Entrez un deuxième entier : "))

if a > b:
    print("plus grand")
else:
    print("plus petit ou égal")
```

Version 2 : trois messages

```
a = int(input("Entrez un premier entier : "))
b = int(input("Entrez un deuxième entier : "))

if a > b:
    print("plus grand")
elif a < b:
    print("plus petit")
else:
    print("égal")
```

Remarque : `elif` se lit « sinon, si ». Les trois cas `a > b`, `a < b` et `a == b` sont testés dans l'ordre ; le dernier `else` correspond forcément à l'égalité.

Exercice 2 Aire d'un rectangle

```
longueur = float(input("Entrez la longueur : "))
largeur = float(input("Entrez la largeur : "))

if longueur >= largeur:
    print("L'aire du rectangle est", longueur * largeur)
else:
    print("Données incohérentes")
```

Remarque : `float` accepte aussi bien `3` que `2.5`. Le test `longueur >= largeur` se fait *avant* tout calcul.

Exercice 3 Contrôle de saisie

```
a = int(input("Entrez a : "))
b = int(input("Entrez b : "))
c = int(input("Entrez c : "))
d = int(input("Entrez d : "))
e = int(input("Entrez e : "))
f = int(input("Entrez f : "))

nb = 0          # nombre de variables non nulles
if a != 0:
    nb = nb + 1
if b != 0:
    nb = nb + 1
if c != 0:
    nb = nb + 1
if d != 0:
    nb = nb + 1
if e != 0:
    nb = nb + 1
if f != 0:
    nb = nb + 1

if nb >= 3:
    print("La saisie est correcte")
else:
    print("La saisie est incorrecte")
```

Remarque : En Python, `True` vaut 1 et `False` vaut 0 : le comptage peut s'écrire en une ligne, `nb = (a != 0) + (b != 0) + (c != 0) + (d != 0) + (e != 0) + (f != 0)`.

Exercice 4 Tri de trois nombres

```
a = float(input("Entrez a : "))
b = float(input("Entrez b : "))
c = float(input("Entrez c : "))

# On amène le plus petit des trois dans a
if a > b:
    tmp = a
    a = b
    b = tmp
if a > c:
    tmp = a
    a = c
    c = tmp
# Il reste à ordonner b et c
if b > c:
    tmp = b
    b = c
    c = tmp

print(a, b, c)
```

Remarque : Après les deux premiers tests, `a` contient le plus petit des trois nombres ; le troisième test ordonne `b` et `c`. L'échange de deux variables passe par une variable temporaire `tmp` ; Python permet aussi d'écrire directement `a, b = b, a`.

Exercice 5 Minimum, maximum et moyenne

Sans boucle (bloc de saisie répété)

```
# La première valeur initialise le minimum, le maximum et la somme
x = float(input("Entrez une valeur : "))
mini = x
maxi = x
somme = x

# Deuxième valeur
x = float(input("Entrez une valeur : "))
if x < mini:
    mini = x
if x > maxi:
    maxi = x
somme = somme + x

# Troisième valeur
x = float(input("Entrez une valeur : "))
if x < mini:
    mini = x
if x > maxi:
    maxi = x
somme = somme + x

# ... même bloc, recopié à l'identique, pour les valeurs 4 à 10 ...

print("Plus petite valeur :", mini)
print("Plus grande valeur :", maxi)
print("Moyenne :", somme / 10)
```

Avec une boucle (à réécrire après le TD n° 2)

```
x = float(input("Entrez une valeur : "))
mini = x
maxi = x
somme = x
for i in range(9):                  # les neuf valeurs suivantes
    x = float(input("Entrez une valeur : "))
    if x < mini:
        mini = x
    if x > maxi:
        maxi = x
    somme = somme + x

print("Plus petite valeur :", mini)
print("Plus grande valeur :", maxi)
print("Moyenne :", somme / 10)
```

Remarque : Les quatre variables sont `x`, `mini`, `maxi` et `somme`. Initialiser `mini` et `maxi` avec la première valeur saisie évite d'avoir à choisir une valeur de départ arbitraire. Le bloc de traitement est strictement le même pour les neuf valeurs suivantes : c'est exactement ce qu'une boucle permet d'écrire une seule fois.

Exercice 6 Distance entre deux points

```python
from math import sqrt

x1 = int(input("Abscisse du premier point : "))
y1 = int(input("Ordonnée du premier point : "))
x2 = int(input("Abscisse du deuxième point : "))
y2 = int(input("Ordonnée du deuxième point : "))

d = sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
print("La distance entre les deux points est", d)
```

Remarque : $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. La fonction `sqrt` provient du module `math` ; on peut aussi écrire `import math` puis `math.sqrt(...)`.

Exercice 7 Triangle rectangle

```python
xa = int(input("Abscisse de A : "))
ya = int(input("Ordonnée de A : "))
xb = int(input("Abscisse de B : "))
yb = int(input("Ordonnée de B : "))
xc = int(input("Abscisse de C : "))
yc = int(input("Ordonnée de C : "))

# Carrés des longueurs des trois côtés (des entiers : pas de racine carrée)
ab = (xb - xa) ** 2 + (yb - ya) ** 2
bc = (xc - xb) ** 2 + (yc - yb) ** 2
ca = (xa - xc) ** 2 + (ya - yc) ** 2

if ab == 0 or bc == 0 or ca == 0:
    print("Deux points sont confondus")
elif ab == bc + ca or bc == ab + ca or ca == ab + bc:
    print("Triangle rectangle")
else:
    print("Pas un triangle rectangle")
```

Remarque : Le triangle est rectangle si le carré du plus grand côté est égal à la somme des carrés des deux autres (théorème de Pythagore). Comme les coordonnées sont entières, les carrés des longueurs sont des entiers : on les compare exactement, sans racine carrée ni arrondi. Le premier test écarte le cas de deux points confondus.

Exercice 8 Calculatrice

```
op = input("Entrez l'opération : ")
a = int(input("Entrez le premier opérande : "))
b = int(input("Entrez le deuxième opérande : "))

if op == "+":
    print("Le résultat de", a, "+", b, "est", a + b)
elif op == "-":
    print("Le résultat de", a, "-", b, "est", a - b)
elif op == "*":
    print("Le résultat de", a, "*", b, "est", a * b)
elif op == "/":
    if b == 0:
        print("Division par zéro impossible")
    else:
        print("Le résultat de", a, "/", b, "est", a // b)
else:
    print("Opération inconnue")
```

Remarque : Le résultat devant être entier, la division utilise le quotient entier `//` (`7 // 2` vaut `3`) ; l'opérateur `/` donnerait un réel (`3.5`). Le test `b == 0` est placé avant la division. Un opérateur inconnu est également traité.

Exercice 9 Équation du second degré

```
from math import sqrt

a = float(input("Entrez a : "))
b = float(input("Entrez b : "))
c = float(input("Entrez c : "))

if a == 0:
    print("a doit être différent de zéro")
else:
    delta = b ** 2 - 4 * a * c
    if delta < 0:
        print("Pas de solution réelle")
    elif delta == 0:
        print("Une solution double :", -b / (2 * a))
    else:
        x1 = (-b - sqrt(delta)) / (2 * a)
        x2 = (-b + sqrt(delta)) / (2 * a)
        print("Deux solutions :", x1, "et", x2)
```

Remarque : Discriminant $\Delta = b^2 - 4ac$: pas de solution réelle si $\Delta < 0$, une solution double $x = -\frac{b}{2a}$ si $\Delta = 0$, deux solutions $x = \frac{-b \pm \sqrt{\Delta}}{2a}$ si $\Delta > 0$.

Exercice 10 Chiffres d'un nombre

```
n = int(input("Entrez un nombre à 4 chiffres : "))

if n < 1000 or n > 9999:
    print("Le nombre ne contient pas 4 chiffres")
else:
    milliers = n // 1000
    centaines = (n // 100) % 10
    dizaines = (n // 10) % 10
    unites = n % 10
    print(milliers, centaines, dizaines, unites)
```

Remarque : Un entier positif a quatre chiffres s'il est compris entre 1000 et 9999. Les chiffres s'obtiennent par divisions entières (`//`) et restes (`%`) successifs. Autre solution : convertir le nombre en chaîne, `s = str(n)`, puis afficher `s[0]`, `s[1]`, `s[2]`, `s[3]`.

Feuille de TD n° 2

Bases de la programmation – Les boucles

Rappels Python

Répéter tant qu'une condition est vraie. La boucle `while` exécute son bloc tant que la condition est vraie ; elle est typiquement utilisée pour une saisie contrôlée :

```
n = int(input("Entrez un entier positif : "))
while n < 0:                                    # tant que n est négatif...
    n = int(input("Entrez un entier positif : "))
```

Répéter un nombre de fois connu. La boucle `for` parcourt les valeurs de `range` : `range(5)` donne 0, 1, 2, 3, 4 et `range(2, 6)` donne 2, 3, 4, 5 (la borne de fin est exclue) :

```
for i in range(1, 4):
    print("Tour numéro", i)      # affiche Tour numéro 1, puis 2, puis 3
```

Afficher sans passer à la ligne. `print("*", end="")` affiche une étoile sans retour à la ligne ; `print()` seul passe à la ligne. Une chaîne peut aussi être répétée : `"*" * 5` vaut `"*****"`.

Exercice 1 Saisie contrôlée

Écrire un programme qui demande à l'utilisateur de lui fournir un nombre entier compris entre 0 et 99 (inclus), et ceci jusqu'à ce que la réponse soit satisfaisante. Le dialogue avec l'utilisateur se présentera ainsi :

```
Donnez un entier entre 0 et 99 : 453
Donnez un entier entre 0 et 99 : 25
Merci pour le nombre 25
```

Exercice 2 Table de multiplication

Écrire un programme qui demande à l'utilisateur un nombre `n` compris entre 2 et 9 (inclus) et qui affiche sa table de multiplication, de $1 \times n$ à $10 \times n$. La saisie du nombre est bloquante : tant que le nombre n'est pas compris entre 2 et 9, le programme redemande une valeur.

Exemple d'exécution :

```
Entrez un nombre entre 2 et 9 : 12
Entrez un nombre entre 2 et 9 : 7
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Exercice 3 Moyenne d'une suite de valeurs

A. Écrire un programme qui saisit une suite de valeurs entières non nulles. À la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies. La saisie s'arrête quand l'utilisateur donne la valeur 0.

B. Écrire un programme qui saisit une suite de valeurs entières non nulles. À la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies, la valeur minimale et la valeur maximale. Les trois valeurs doivent être calculées en utilisant une seule boucle. La saisie s'arrête quand l'utilisateur donne la valeur 0.

Dans les deux cas, la valeur 0 qui termine la saisie n'est pas prise en compte. Si l'utilisateur tape 0 dès le départ, le programme affiche « Aucune valeur saisie ».

Exercice 4 Moyennes des valeurs paires et impaires

Écrire un programme qui saisit une suite de valeurs entières non nulles. À la fin de la saisie, le programme doit afficher la moyenne des valeurs paires et la moyenne des valeurs impaires. La saisie s'arrête quand l'utilisateur donne la valeur 0.

Si aucune valeur paire (respectivement impaire) n'a été saisie, le programme affiche « Aucune valeur paire » (respectivement « Aucune valeur impaire ») à la place de la moyenne correspondante.

> **Attention :** la division par zéro est une opération interdite.

Exercice 5 Le juste prix

A. Écrire un programme qui permet de déterminer le juste prix (juste prix = 42). L'utilisateur saisit un nombre entier. Le programme répond « inférieur » (le juste prix est inférieur au prix proposé) si le prix indiqué est supérieur à 42, il répond « supérieur » (le juste prix est supérieur au prix proposé) si le prix indiqué est inférieur à 42, et il répond « BRAVO ! » s'il trouve le prix exact. Tant que le prix n'est pas le bon, l'utilisateur doit retenter sa chance en saisissant un nouveau nombre.

Exemple d'exécution :

```
Votre prix : 50
inférieur
Votre prix : 30
supérieur
Votre prix : 42
BRAVO !
```

B. Modifier le programme précédent afin que le juste prix soit tiré au hasard entre 1 et 100 (inclus), et non codé en dur à 42 ; il sera évidemment inconnu de l'utilisateur.

De plus, le programme doit, à la fin de chaque partie, afficher le nombre de coups de la partie (un coup est une proposition de l'utilisateur) et demander à l'utilisateur s'il veut continuer le jeu. Si la réponse est exactement « o », le programme lance une nouvelle partie ; sinon, il affiche le nombre de coups moyen sur l'ensemble des parties jouées, puis « À bientôt », et s'arrête.

> **Indication :** après `from random import randint`, l'appel `randint(1, 100)` renvoie un entier tiré au hasard entre 1 et 100 (inclus).

Exercice 6 Rectangle d'étoiles

Le programme a besoin de deux entrées, que l'utilisateur doit fournir : la largeur (nombre de colonnes) et la hauteur (nombre de lignes) du rectangle.

A. Écrire un programme qui dessine un rectangle *plein* d'étoiles `*`.

Exemple d'exécution :

```
Largeur : 5
Hauteur : 3
*****
*****
*****
```

B. Modifier le programme pour qu'il dessine un rectangle *vide* : seules les étoiles du contour sont affichées, l'intérieur est rempli d'espaces.

Exemple d'exécution :

```
Largeur : 5
Hauteur : 4
*****
*   *
*   *
*****
```

Exercice 7 Dessiner un X

Écrire un programme qui permet de dessiner un `X` avec des `*`. Le programme a besoin d'une entrée : la taille de la forme, qui occupe `taille` lignes et `taille` colonnes. La taille doit être un nombre entier impair compris entre 15 et 20 (inclus), c'est-à-dire 15, 17 ou 19. Tant que le nombre saisi ne vérifie pas la condition, le programme redemande la valeur de la taille.

Exemple pour une taille de 5 (la taille demandée est plus grande) :

```
*   *
 * *
  *
 * *
*   *
```

Exercice 8 Factorielle

Écrire un programme qui calcule et affiche la factorielle d'un entier $n \geq 0$:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1\,;$$

par définition, $0! = 1$. Si la valeur saisie est négative, le programme la redemande.

Exercice 9 Nombres premiers

Un entier $n \geq 2$ est dit premier si ses seuls diviseurs sont 1 et lui-même ; 1 n'est pas premier. Écrire un programme qui saisit un entier et qui affiche « PREMIER » s'il est premier et « PAS PREMIER » sinon.

Modifier le programme pour qu'il saisisse deux entiers positifs `a` et `b`, dans n'importe quel ordre, et qu'il affiche tous les nombres premiers compris entre les deux, bornes incluses.

Feuille de TD n° 2 – Corrigé

Bases de la programmation – Les boucles

Remarque générale : les solutions n'utilisent que les boucles `while` et `for`, les instructions conditionnelles et les notions de la feuille de TD n° 1. Les saisies qui se répètent suivent toujours le même schéma : une première lecture avant la boucle, puis une nouvelle lecture à la fin de chaque tour. D'autres solutions correctes sont bien sûr possibles.

Exercice 1 Saisie contrôlée

```
n = int(input("Donnez un entier entre 0 et 99 : "))
while n < 0 or n > 99:
    n = int(input("Donnez un entier entre 0 et 99 : "))
print("Merci pour le nombre", n)
```

Remarque : La condition de la boucle est la *négation* de ce que l'on veut obtenir : on redemande tant que le nombre est strictement négatif *ou* strictement supérieur à 99.

Exercice 2 Table de multiplication

```
n = int(input("Entrez un nombre entre 2 et 9 : "))
while n < 2 or n > 9:
    n = int(input("Entrez un nombre entre 2 et 9 : "))

for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Remarque : `range(1, 11)` parcourt les entiers de 1 à 10 : la borne de fin est exclue.

Exercice 3 Moyenne d'une suite de valeurs

Partie A

```
somme = 0
nb = 0            # nombre de valeurs saisies
x = int(input("Entrez une valeur (0 pour terminer) : "))
while x != 0:
    somme = somme + x
    nb = nb + 1
    x = int(input("Entrez une valeur (0 pour terminer) : "))

if nb == 0:
    print("Aucune valeur saisie")
else:
    print("Moyenne :", somme / nb)
```

Partie B

```
somme = 0
nb = 0
x = int(input("Entrez une valeur (0 pour terminer) : "))
mini = x          # la première valeur initialise le minimum et le maximum
maxi = x
while x != 0:
    somme = somme + x
    nb = nb + 1
    if x < mini:
        mini = x
    if x > maxi:
        maxi = x
    x = int(input("Entrez une valeur (0 pour terminer) : "))

if nb == 0:
    print("Aucune valeur saisie")
else:
    print("Moyenne :", somme / nb)
    print("Minimum :", mini)
    print("Maximum :", maxi)
```

Remarque : La valeur 0 qui termine la saisie n'est pas comptée : elle est lue, puis la condition `x != 0` arrête la boucle avant tout traitement. Le test `nb == 0` évite la division par zéro si l'utilisateur tape 0 dès le départ. Dans la partie B, initialiser `mini` et `maxi` avec la première valeur lue évite de choisir une valeur de départ arbitraire ; si cette valeur est 0, elles ne sont jamais affichées.

Exercice 4 Moyennes des valeurs paires et impaires

```
somme_pairs = 0
nb_pairs = 0
somme_impairs = 0
nb_impairs = 0
x = int(input("Entrez une valeur (0 pour terminer) : "))
while x != 0:
    if x % 2 == 0:
        somme_pairs = somme_pairs + x
        nb_pairs = nb_pairs + 1
    else:
        somme_impairs = somme_impairs + x
        nb_impairs = nb_impairs + 1
    x = int(input("Entrez une valeur (0 pour terminer) : "))

if nb_pairs == 0:
    print("Aucune valeur paire")
else:
    print("Moyenne des valeurs paires :", somme_pairs / nb_pairs)

if nb_impairs == 0:
    print("Aucune valeur impaire")
else:
    print("Moyenne des valeurs impaires :", somme_impairs / nb_impairs)
```

Remarque : Une valeur est paire si le reste de sa division par 2 est nul (`x % 2 == 0`), y compris pour les valeurs négatives. Chaque moyenne a sa propre somme et son propre compteur, et chaque compteur est testé avant la division.

Exercice 5 Le juste prix

Partie A

```
juste_prix = 42
prix = int(input("Votre prix : "))
while prix != juste_prix:
    if prix > juste_prix:
        print("inférieur")
    else:
        print("supérieur")
    prix = int(input("Votre prix : "))
print("BRAVO !")
```

Partie B

```
from random import randint

total_coups = 0   # nombre de coups cumulé sur toutes les parties
nb_parties = 0
continuer = "o"
while continuer == "o":
    juste_prix = randint(1, 100)   # inconnu de l'utilisateur
    coups = 1
    prix = int(input("Votre prix : "))
    while prix != juste_prix:
        if prix > juste_prix:
            print("inférieur")
        else:
            print("supérieur")
        prix = int(input("Votre prix : "))
        coups = coups + 1
    print("BRAVO ! Prix trouvé en", coups, "coups")

    nb_parties = nb_parties + 1
    total_coups = total_coups + coups
    continuer = input("Voulez-vous continuer (o/n) ? ")

print("Nombre de coups moyen :", total_coups / nb_parties)
print("À bientôt")
```

Remarque : Deux boucles imbriquées : la boucle extérieure enchaîne les parties tant que la réponse est « o », la boucle intérieure est celle de la partie A. Le compteur `coups` part de 1 car la première proposition est un coup. Le nombre moyen de coups vaut `total_coups / nb_parties` ; `nb_parties` est toujours au moins 1 puisque `continuer` vaut `"o"` au départ.

Exercice 6 Rectangle d'étoiles

Partie A : rectangle plein

```
largeur = int(input("Largeur : "))
hauteur = int(input("Hauteur : "))

for i in range(hauteur):          # une ligne à chaque tour
    for j in range(largeur):      # largeur étoiles sur la ligne
        print("*", end="")
    print()                       # retour à la ligne
```

Partie B : rectangle vide

```
largeur = int(input("Largeur : "))
hauteur = int(input("Hauteur : "))

for i in range(hauteur):          # ligne i
    for j in range(largeur):      # colonne j
        # une case du contour est sur la première ou la dernière ligne,
        # ou sur la première ou la dernière colonne
        if i == 0 or i == hauteur - 1 or j == 0 or j == largeur - 1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Remarque : Dans les deux cas, la boucle intérieure dessine une ligne et `print()` passe à la ligne suivante. Pour le rectangle plein, la boucle intérieure peut être remplacée par `print("*" * largeur)`. Pour le rectangle vide, la case (`i`, `j`) est une étoile si elle est sur le bord : première ou dernière ligne, première ou dernière colonne ; les cas limites (largeur ou hauteur égale à 1) sont traités naturellement, toutes les cases étant sur le bord.

Exercice 7 Dessiner un X

```
taille = int(input("Taille (entier impair entre 15 et 20) : "))
while taille < 15 or taille > 20 or taille % 2 == 0:
    taille = int(input("Taille (entier impair entre 15 et 20) : "))

for i in range(taille):                  # ligne i
    for j in range(taille):              # colonne j
        if j == i or j == taille - 1 - i:  # les deux diagonales
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Remarque : On parcourt la grille ligne par ligne et colonne par colonne. La case (`i`, `j`) est sur la première diagonale si `j == i`, sur la seconde si `j == taille - 1 - i` ; les deux diagonales se croisent au centre parce que la taille est impaire. La condition de la saisie regroupe les trois contraintes : au moins 15, au plus 20, impair.

Exercice 8 Factorielle

```python
n = int(input("Entrez un entier positif : "))
while n < 0:
    n = int(input("Entrez un entier positif : "))

fact = 1
for i in range(2, n + 1):
    fact = fact * i
print("La factorielle de", n, "est", fact)
```

Remarque : Le produit part de 1 et `range(2, n + 1)` multiplie successivement par 2, 3, …, n. Pour $n = 0$ ou $n = 1$, la boucle ne fait aucun tour et le résultat reste 1, conformément à la définition.

Exercice 9 Nombres premiers

Un entier

```python
n = int(input("Entrez un entier : "))

nb_diviseurs = 0
for d in range(1, n + 1):
    if n % d == 0:
        nb_diviseurs = nb_diviseurs + 1

if nb_diviseurs == 2:       # exactement deux diviseurs : 1 et n
    print("PREMIER")
else:
    print("PAS PREMIER")
```

Tous les nombres premiers entre deux entiers

```python
a = int(input("Entrez le premier entier : "))
b = int(input("Entrez le deuxième entier : "))
if a > b:                   # on s'assure que a <= b
    a, b = b, a

for n in range(a, b + 1):
    nb_diviseurs = 0
    for d in range(1, n + 1):
        if n % d == 0:
            nb_diviseurs = nb_diviseurs + 1
    if nb_diviseurs == 2:
        print(n)
```

Remarque : Un nombre est premier s'il possède exactement deux diviseurs, 1 et lui-même : cette version compte les diviseurs, ce qui écarte naturellement 0, 1 et les nombres négatifs. Dans la seconde partie, l'échange `a, b = b, a` garantit `a <= b` quel que soit l'ordre de saisie, et `range(a, b + 1)` inclut les deux bornes. Pour aller plus vite, on peut arrêter la recherche dès qu'un diviseur autre que 1 est trouvé, et ne tester que les diviseurs `d` tels que `d * d <= n`.

Feuille de TD n° 3

Bases de la programmation – Les fonctions

Rappels Python

Définir et appeler une fonction. Une fonction se définit avec `def`, ses paramètres entre parenthèses et un bloc indenté ; `return` renvoie une valeur à l'appelant et termine immédiatement la fonction :

```
def aire_rectangle(largeur, hauteur):    # définition
    aire = largeur * hauteur
    return aire                          # valeur renvoyée

a = aire_rectangle(3, 4)                 # appel : a vaut 12
print(aire_rectangle(2, 5))              # affiche 10
```

Renvoyer n'est pas afficher. `print` affiche à l'écran mais ne renvoie rien ; une fonction qui doit « donner » un résultat utilise `return`. Une fonction peut renvoyer un booléen ou plusieurs valeurs à la fois :

```
def est_pair(n):
    return n % 2 == 0                    # renvoie True ou False

def division(a, b):
    return a // b, a % b                 # renvoie deux valeurs

q, r = division(17, 5)                   # q vaut 3, r vaut 2
```

Dans cette feuille, les boucles s'écrivent uniquement avec `while`.

Exercice 1 Surface d'un cercle

Définir une fonction `surfCercle(R)` qui renvoie la surface (l'aire) d'un cercle dont le rayon `R` est fourni en argument. Par exemple, l'instruction `print(surfCercle(2.5))` doit afficher 19.634954… (soit 19.635 à 10^{-3} près).

Exercice 2 Maximum de quatre nombres

Définir une fonction `maximum(n1, n2, n3, n4)` qui renvoie le plus grand des quatre nombres fournis en arguments. La fonction ne fait aucun affichage : c'est le programme qui l'appelle qui affiche le résultat. Par exemple, l'instruction `print("La valeur maximale est :", maximum(1, 5, 4, 6))` doit afficher « La valeur maximale est : 6 ».

Exercice 3 Équation du second degré

On souhaite résoudre l'équation $ax^2 + bx + c = 0$ pour tous réels `a`, `b` et `c`, y compris lorsque `a` ou `b` sont nuls. La méthode est la suivante (S désigne l'ensemble des solutions) :

Entrée : trois réels a, b et c
Sortie : l'ensemble S des solutions de $ax^2 + bx + c = 0$

si $a = 0$ **alors**
  si $b = 0$ **alors**
    si $c = 0$ **alors** $S \leftarrow \mathbb{R}$ (tous les réels sont solutions)
    sinon $S \leftarrow \emptyset$ (pas de solution)
  sinon
    $S \leftarrow \{-c/b\}$
sinon
  $\Delta \leftarrow b^2 - 4ac$
  si $\Delta = 0$ **alors** $S \leftarrow \{-b/(2a)\}$
  sinon si $\Delta > 0$ **alors** $S \leftarrow \{\dots, \dots\}$ (à compléter)
  sinon $S \leftarrow \emptyset$ (pas de solution réelle)

A. Écrire la fonction `discriminant(a, b, c)` qui renvoie le discriminant $\Delta = b^2 - 4ac$.

B. Écrire la fonction `resoudre(a, b, c)` qui résout l'équation. **Attention :** cette fonction ne doit faire aucun affichage. Elle renvoie trois valeurs `nb, x1, x2` : `nb` est le nombre de solutions (-1 si tous les réels sont solutions, sinon 0, 1 ou 2) et `x1`, `x2` sont les solutions éventuelles (`x1 < x2` s'il y en a deux ; une valeur inutilisée vaut `None`).

C. Écrire une fonction d'affichage, sans paramètre, qui demande `a`, `b` et `c` à l'utilisateur, appelle `resoudre` puis affiche le résultat de la résolution. Exemple d'exécution :

```
Entrez a : 0
Entrez b : 2
Entrez c : 1
Votre équation admet une seule solution : -0.5
```

Exercice 4 Cours d'une action

A. Une action est cotée à 57 €. Sa valeur augmente de 3 % tous les mois. Écrire une fonction sans paramètre qui renvoie le nombre de mois à attendre pour que sa valeur dépasse (strictement) 200 €.

B. Proposer une généralisation de la fonction précédente : étant donnés le prix initial d'une action, son taux d'augmentation mensuel (0.03 dans la question A) et une valeur cible, la fonction renvoie le nombre de mois à attendre pour que la valeur de l'action dépasse la valeur cible.

Exercice 5 Somme des chiffres

Écrire une fonction `somme` qui prend en argument un entier positif et qui renvoie la somme des chiffres qui le composent. Par exemple, `print(somme(1234))` affiche 10.

Exercice 6 Année bissextile

Écrire une fonction `bissextile` qui prend en argument un entier représentant une année et qui renvoie un booléen indiquant si l'année est bissextile. On rappelle qu'une année est bissextile si elle

possède un 29 février, c'est-à-dire si elle est multiple de 4 mais pas de 100, ou si elle est multiple de 400.

Exemples : `bissextile(2024)` renvoie `True`, `bissextile(1800)` renvoie `False` et `bissextile(2000)` renvoie `True`.

Exercice 7 Comparaison de réels et triangle rectangle

A. Écrire une fonction `identiques(x, y, epsilon)` qui détermine si deux nombres réels sont identiques à `epsilon` près, c'est-à-dire si $|x - y| <$ `epsilon`. Exemple : 2.01 et 2.005 sont identiques à 0.01 près, car $|2.01 - 2.005| = 0.005 < 0.01$.

Dans la suite, on considérera que deux réels sont identiques s'ils le sont à 10^{-6} près, c'est-à-dire avec `epsilon` $= 10^{-6}$.

B. Écrire une fonction qui prend en argument trois nombres réels strictement positifs et qui renvoie `True` ou `False` selon qu'ils peuvent représenter les carrés des longueurs des côtés d'un triangle rectangle, c'est-à-dire si le plus grand est identique (au sens de la question A) à la somme des deux autres.

C. Écrire une fonction qui prend en argument quatre nombres réels strictement positifs et qui renvoie `True` si trois d'entre eux peuvent représenter les carrés des longueurs des côtés d'un triangle rectangle, `False` sinon. On utilisera la fonction de la question B sur les quatre triplets possibles.

Exercice 8 Décomposition en facteurs premiers

Voici la méthode de décomposition d'un entier en facteurs premiers.

Entrée : un entier naturel $n > 1$
Sortie : une chaîne de caractères qui représente la décomposition de n en facteurs premiers

`resultat` ← chaîne vide
tant que $n > 1$ **faire**
  d ← le plus petit diviseur premier de n
  si `resultat` est vide **alors**
    mettre d dans `resultat`
  sinon
    ajouter `" x "` puis d à la fin de `resultat`
  $n \leftarrow n \,/\, d$ (division entière)
retourner `resultat`

A. Écrire une fonction `est_premier(n)` qui renvoie `True` si `n` est un nombre premier, `False` sinon.

B. Écrire une fonction `plus_petit_diviseur(n)` qui utilise `est_premier` et renvoie le plus petit diviseur premier de `n` (strictement supérieur à 1).

C. Écrire une fonction `decomposition_facteurs(n)` qui produit la décomposition en facteurs premiers de `n` en appelant plusieurs fois la fonction de la question B. Par exemple, pour $n = 12$, la fonction renvoie la chaîne `"2 x 2 x 3"`.

> **Indication :** ici, exceptionnellement, on utilise une variable de type `str` (chaîne de caractères). Au début, on écrit `resultat = ""`. Si `resultat` vaut à un moment `"2 x 2"` et que l'on veut ajouter l'entier `d` comme facteur, on écrit simplement `resultat = resultat + " x " + str(d)`.

Feuille de TD n° 3 – Corrigé

Bases de la programmation – Les fonctions

Remarque générale : conformément à l'énoncé, les boucles sont écrites uniquement avec `while`. Chaque programme se termine par un ou plusieurs appels qui montrent l'utilisation de la fonction ; le résultat attendu figure en commentaire. D'autres solutions correctes sont bien sûr possibles.

Exercice 1 Surface d'un cercle

```python
from math import pi

def surfCercle(R):
    return pi * R ** 2

print(surfCercle(2.5))      # affiche 19.634954084936208
```

Remarque : La constante `pi` provient du module `math`. La fonction renvoie la valeur exacte calculée par Python ; l'arrondi à 19.635 n'est qu'une lecture du résultat.

Exercice 2 Maximum de quatre nombres

```python
def maximum(n1, n2, n3, n4):
    m = n1                  # le plus grand rencontré jusqu'ici
    if n2 > m:
        m = n2
    if n3 > m:
        m = n3
    if n4 > m:
        m = n4
    return m

print("La valeur maximale est :", maximum(1, 5, 4, 6))
```

Remarque : La variable `m` contient à tout moment le plus grand des nombres déjà examinés. Le message « La valeur maximale est : » est écrit par l'appelant, pas par la fonction, qui se contente de renvoyer la valeur.

Exercice 3 Équation du second degré

```python
from math import sqrt

def discriminant(a, b, c):
    return b ** 2 - 4 * a * c

def resoudre(a, b, c):
```

```
    # renvoie nb, x1, x2 :
    #   nb = -1 : tous les réels sont solutions ; nb = 0 : aucune solution
    #   nb = 1  : une solution x1 ; nb = 2 : deux solutions x1 < x2
    if a == 0:
        if b == 0:
            if c == 0:
                return -1, None, None
            else:
                return 0, None, None
        else:
            return 1, -c / b, None
    else:
        delta = discriminant(a, b, c)
        if delta == 0:
            return 1, -b / (2 * a), None
        elif delta > 0:
            x1 = (-b - sqrt(delta)) / (2 * a)
            x2 = (-b + sqrt(delta)) / (2 * a)
            if x1 > x2:             # si a < 0, l'ordre est inversé
                x1, x2 = x2, x1
            return 2, x1, x2
        else:
            return 0, None, None

def afficher_solutions():
    a = float(input("Entrez a : "))
    b = float(input("Entrez b : "))
    c = float(input("Entrez c : "))
    nb, x1, x2 = resoudre(a, b, c)
    if nb == -1:
        print("Tous les réels sont solutions de votre équation")
    elif nb == 0:
        print("Votre équation n'admet aucune solution réelle")
    elif nb == 1:
        print("Votre équation admet une seule solution :", x1)
    else:
        print("Votre équation admet deux solutions :", x1, "et", x2)

afficher_solutions()
```

Remarque : `resoudre` ne fait aucun affichage : elle renvoie un triplet, et c'est `afficher_solutions` qui traduit ce triplet en phrase. Chaque `return` termine immédiatement la fonction. Lorsque $a < 0$, la formule donne $x_1 > x_2$: l'échange final garantit `x1 < x2`. L'exemple de l'énoncé ($a = 0$, $b = 2$, $c = 1$) passe par la branche `S = {-c/b}` et affiche -0.5.

Exercice 4 Cours d'une action

```python
def nb_mois_200():
    valeur = 57
    mois = 0
    while valeur <= 200:
        valeur = valeur * 1.03      # + 3 %
        mois = mois + 1
    return mois

def nb_mois(prix_initial, taux, valeur_cible):
    valeur = prix_initial
    mois = 0
    while valeur <= valeur_cible:
        valeur = valeur * (1 + taux)
        mois = mois + 1
    return mois

print(nb_mois_200())                # 43
print(nb_mois(57, 0.03, 200))       # 43
```

Remarque : On répète « un mois de plus » tant que la valeur n'a pas dépassé la cible : la condition du `while` est `valeur <= valeur_cible` (négation de « dépasse strictement »). Augmenter de 3 % revient à multiplier par 1,03. La question A correspond à l'appel `nb_mois(57, 0.03, 200)` : 43 mois ($57 \times 1{,}03^{42} \approx 197{,}26$ et $57 \times 1{,}03^{43} \approx 203{,}18$).

Exercice 5 Somme des chiffres

```python
def somme(n):
    s = 0
    while n > 0:
        s = s + n % 10      # dernier chiffre de n
        n = n // 10         # on retire ce chiffre
    return s

print(somme(1234))          # 10
```

Remarque : `n % 10` est le chiffre des unités et `n // 10` supprime ce chiffre ; la boucle s'arrête quand il ne reste plus de chiffre. Pour $n = 0$, la boucle ne fait aucun tour et la fonction renvoie 0.

Exercice 6 Année bissextile

```python
def bissextile(annee):
    return (annee % 4 == 0 and annee % 100 != 0) or annee % 400 == 0

print(bissextile(2024), bissextile(1800), bissextile(2000))    # True False True
```

Remarque : La condition est directement la règle de l'énoncé : une expression booléenne peut être renvoyée telle quelle, sans `if` … `return True` … `else` … `return False`.

Exercice 7 Comparaison de réels et triangle rectangle

```python
def identiques(x, y, epsilon):
    return abs(x - y) < epsilon

def triangle_rectangle(c1, c2, c3):
    # c1, c2, c3 : carrés des longueurs des côtés
    # rectangle si le plus grand carré est la somme des deux autres (Pythagore)
    return (identiques(c1, c2 + c3, 1e-6) or identiques(c2, c1 + c3, 1e-6)
            or identiques(c3, c1 + c2, 1e-6))

def triangle_rectangle_4(c1, c2, c3, c4):
    # les quatre triplets possibles
    return (triangle_rectangle(c1, c2, c3) or triangle_rectangle(c1, c2, c4)
            or triangle_rectangle(c1, c3, c4) or triangle_rectangle(c2, c3, c4))

print(identiques(2.01, 2.005, 0.01))        # True
print(triangle_rectangle(9, 16, 25))        # True
print(triangle_rectangle(9, 16, 24))        # False
print(triangle_rectangle_4(1, 25, 9, 16))   # True
```

Remarque : Deux réels calculés ne sont presque jamais exactement égaux ($0.1 + 0.2$ vaut 0.30000000000000004 en Python), d'où la comparaison à `epsilon` près avec `abs`. La question B teste les trois façons de choisir « le plus grand » sans avoir à le déterminer, et la question C réutilise B sur les quatre triplets que l'on peut former en écartant l'un des quatre nombres.

Exercice 8 Décomposition en facteurs premiers

```python
def est_premier(n):
    if n < 2:
        return False
    d = 2
    while d < n:
        if n % d == 0:      # un diviseur autre que 1 et n
            return False
        d = d + 1
    return True

def plus_petit_diviseur(n):
    d = 2
    while n % d != 0 or not est_premier(d):
        d = d + 1
    return d

def decomposition_facteurs(n):
    resultat = ""
    while n > 1:
        d = plus_petit_diviseur(n)
        if resultat == "":
            resultat = str(d)
        else:
            resultat = resultat + " x " + str(d)
        n = n // d
    return resultat

print(decomposition_facteurs(12))     # 2 x 2 x 3
print(decomposition_facteurs(97))     # 97
print(decomposition_facteurs(360))    # 2 x 2 x 2 x 3 x 3 x 5
```

Remarque : Dans `est_premier`, le `return False` placé dans la boucle arrête la recherche dès le premier diviseur trouvé ; on peut aussi limiter la boucle à `d * d <= n`. Dans `plus_petit_diviseur`, le test `est_premier(d)` est demandé par l'énoncé mais il est en fait toujours vrai pour le premier `d` qui divise `n` (le plus petit diviseur supérieur à 1 d'un entier est nécessairement premier). `str(d)` convertit l'entier en chaîne avant la concaténation avec `+`.

Feuille de TD n° 4

Bases de la programmation – Les listes de listes

Rappels Python

Listes. Une liste se crée avec des crochets ; `L.append(x)` ajoute `x` à la fin de `L`, `L[i]` désigne l'élément d'indice `i` (le premier a l'indice 0) et `len(L)` est la taille de la liste. Une matrice se représente par une liste de listes : chaque ligne est une sous-liste.

```
M = [[1, 2, 3], [4, 5, 6]]      # matrice de 2 lignes et 3 colonnes
print(M[1])                     # [4, 5, 6] : la ligne d'indice 1
print(M[1][2])                  # 6 : ligne 1, colonne 2
M[0][0] = 9                     # modifie un élément
L = []                          # liste vide
L.append(7)                     # L vaut [7]
print(len(L))                   # 1
```

Une liste passée en argument peut être modifiée par la fonction. Les modifications faites sur ses éléments sont visibles par l'appelant, même sans `return` :

```
def ajouter_un(L):
    L[0] = L[0] + 1             # modifie la liste de l'appelant

T = [5, 6]
ajouter_un(T)
print(T)                        # [6, 6]
```

Nombres aléatoires. Après `from random import randint`, `randint(a, b)` renvoie un entier tiré au hasard entre `a` et `b` (inclus).

Exercice 1 Matrice nulle

En Python, une matrice peut être représentée par une liste de listes : chaque ligne de la matrice est une sous-liste. Par exemple, la matrice ci-dessous, à $n = 5$ lignes et $m = 4$ colonnes, est une liste de 5 sous-listes contenant chacune 4 entiers :

```
[[1, 2, 3, 4],
 [4, 5, 6, 7],
 [8, 9, 1, 4],
 [1, 2, 5, 4],
 [1, 2, 2, 4]]
```

Écrire une fonction `zeros(n, m)` qui crée et renvoie une matrice de `n` lignes et `m` colonnes initialisée avec des 0, c'est-à-dire une liste de `n` sous-listes contenant chacune `m` entiers égaux à 0. On utilisera la méthode `append` en partant de listes vides.

Exemple : `zeros(2, 3)` renvoie `[[0, 0, 0], [0, 0, 0]]`.

Exercice 2 Matrice aléatoire

Écrire une fonction `aleatoire(n, m, a, b)` qui crée et renvoie une matrice de `n` lignes et `m` colonnes dont chaque élément est un entier tiré au hasard entre `a` et `b` (inclus).

Convention : à partir de maintenant, toute fonction prenant en argument une matrice prend aussi en argument ses dimensions, `n` (nombre de lignes) puis `m` (nombre de colonnes) : `f(M, n, m, ...)`.

Exercice 3 Affichage

Écrire une fonction `afficher(M, n, m)` qui affiche la matrice `M` ligne par ligne, les valeurs d'une même ligne étant séparées par un espace. Pour la matrice de l'exercice 1, l'affichage est :

```
1 2 3 4
4 5 6 7
8 9 1 4
1 2 5 4
1 2 2 4
```

Exercice 4 Occurrences d'une valeur

Écrire une fonction `occurrences(M, n, m, v)` qui renvoie le nombre de fois où la valeur `v` apparaît dans la matrice `M` (le nombre d'occurrences de `v`).

Exercice 5 Deux lignes identiques

Écrire une fonction `lignes_identiques(M, n, m, i, j)` qui renvoie `True` si les lignes d'indices `i` et `j` de la matrice `M` sont identiques (mêmes valeurs aux mêmes positions), `False` sinon. On comparera les éléments un à un, sans utiliser l'opérateur `==` entre deux listes : la même méthode servira pour les colonnes.

Exercice 6 Existence de deux lignes identiques

Écrire une fonction `deux_lignes_identiques(M, n, m)` qui renvoie `True` s'il existe dans la matrice `M` deux lignes distinctes identiques, `False` sinon. On utilisera la fonction de l'exercice 5.

Exercice 7 Colonnes identiques

Même travail pour les colonnes, en suivant la méthode des exercices 5 et 6 : écrire une fonction `colonnes_identiques(M, n, m, j, k)` qui renvoie `True` si les colonnes d'indices `j` et `k` sont identiques, puis une fonction `deux_colonnes_identiques(M, n, m)` qui renvoie `True` s'il existe deux colonnes distinctes identiques.

Exercice 8 Produit de deux matrices

Soit `A` une matrice de dimensions $n \times m$ et `B` une matrice de dimensions $m \times p$ (le produit n'est défini que si le nombre de colonnes de `A` est égal au nombre de lignes de `B`). Le produit $C = A \times B$ est la matrice de dimensions $n \times p$ définie par

$$C[i][j] = \sum_{k=0}^{m-1} A[i][k] \times B[k][j].$$

Écrire une fonction `produit(A, n, m, B, p)` qui renvoie la matrice `C` ainsi que ses dimensions `n` et `p` (trois valeurs). On pourra créer `C` avec la fonction `zeros` de l'exercice 1.

Exercice 9 Matrice symétrique

Écrire une fonction `symetrique(M, n, m)` qui renvoie `True` si la matrice `M` est symétrique, `False` sinon. Une matrice est symétrique si elle est carrée ($n = m$) et si $M[i][j] = M[j][i]$ pour tous `i` et `j`.

Exercice 10 Liste et matrice triées

A. Une liste `L` de taille `n` est dite triée si et seulement si, pour tout $i < n - 1$, $L[i] \leq L[i + 1]$: les valeurs sont rangées dans l'ordre croissant, des valeurs égales étant autorisées. Écrire une fonction `liste_triee(L)` qui renvoie `True` si la liste `L` est triée et `False` sinon.

B. Une matrice est dite triée si et seulement si toutes ses lignes sont triées. Écrire une fonction `matrice_triee(M, n, m)` qui renvoie `True` si la matrice `M` est triée et `False` sinon.

Exercice 11 Tri d'une liste

A. Écrire une fonction `passe(L, n)` qui prend en arguments une liste `L` et un entier `n` (inférieur ou égal à la taille de la liste) et qui, pour tout $i < n - 1$ pris dans l'ordre croissant, échange les valeurs de `L[i]` et `L[i+1]` lorsque `L[i] > L[i+1]`.

- S'il n'existe aucun `i` tel que `L[i] > L[i+1]` (aucun échange n'a été nécessaire), la fonction renvoie `True`.
- Sinon, elle renvoie `False`.

La liste est modifiée directement par la fonction.

B. Utiliser cette fonction pour écrire une fonction `tri(L, n)` qui trie la liste `L` : on répète des passes tant que la fonction précédente renvoie `False`.

Exercice 12 Tri d'une matrice

Écrire une fonction `trier_matrice(M, n, m)` qui trie une matrice, c'est-à-dire chacune de ses lignes, à l'aide de la fonction `tri` de l'exercice 11.

Feuille de TD n° 5

Bases de la programmation – Révision épreuve écrite

Objectif : ce TD vise à tester la compréhension des structures de contrôle, des fonctions, des listes et de la logique algorithmique en Python, dans les conditions de l'épreuve écrite.
Consignes : les seules opérations autorisées sur les listes sont la création d'une liste vide `[]`, `len(L)`, l'accès `L[i]`, `L.append(x)` et le parcours `for x in L`. On n'utilisera ni l'opérateur `in`, ni les méthodes `sort`, `count` et `index`, ni les fonctions `sorted`, `min`, `max` et `sum` : l'objectif est d'écrire soi-même les boucles. Les fonctions d'un exercice peuvent être réutilisées dans les exercices suivants.

Exercice 1 Analyse de programme

Voici un extrait de code :

```
for x in range(1, 10):
    print(x, ": ", end="")
    if x % 2 == 0:
        print("pair")
    elif x % 3 == 0:
        print("triple")
    else:
        print("impair")
```

Expliquer brièvement ce que fait ce programme et indiquer les messages qu'il affiche. Que se passe-t-il pour $x = 6$? Pourquoi ?

Exercice 2 Nombre abondant

Un nombre entier $n \geq 1$ est dit *abondant* si la somme de ses diviseurs propres (ses diviseurs strictement inférieurs à lui-même, 1 compris) est strictement supérieure à lui-même.

Écrire une fonction `abondant(n)` qui renvoie `True` si `n` est abondant, `False` sinon.

```
abondant(12)    # True, car 1 + 2 + 3 + 4 + 6 = 16 > 12
abondant(28)    # False, car 1 + 2 + 4 + 7 + 14 = 28
```

Exercice 3 Liste des nombres abondants

Écrire une fonction `abondants_liste(L)` qui renvoie une nouvelle liste contenant uniquement les nombres abondants de la liste `L`, dans le même ordre.

```
abondants_liste([10, 12, 18, 20])    # [12, 18, 20]
abondants_liste([6, 28, 496])        # []
```

Exercice 4 Position du minimum

Écrire une fonction `position_min(L)` qui renvoie l'indice du plus petit élément de la liste `L`. On suppose que la liste contient au moins un élément ; si le minimum apparaît plusieurs fois, la fonction renvoie l'indice de sa première occurrence.

```
position_min([4, 2, 7, 1, 3])    # 3
position_min([10, 5, 5, 8])      # 1
position_min([42])               # 0
```

Exercice 5 Tri par sélection

Écrire une fonction `tri_selection(L)` qui trie une liste d'entiers `L` dans l'ordre croissant en utilisant l'algorithme du tri par sélection, puis renvoie la liste triée. Rappel de l'algorithme : pour chaque position `i` de 0 à $n-2$, on cherche la position du plus petit élément parmi `L[i]`, …, `L[n - 1]` et on l'échange avec `L[i]`.

```
tri_selection([5, 2, 9, 1])    # [1, 2, 5, 9]
tri_selection([3, 3, 2])       # [2, 3, 3]
```

Exercice 6 Doublons optimisés

Écrire une fonction `doublon(L)` qui renvoie `True` si la liste contient au moins un doublon (une valeur présente au moins deux fois), `False` sinon. La fonction ne doit pas faire plus de $n(n-1)/2$ comparaisons, `n` étant la taille de la liste : chaque paire d'éléments n'est comparée qu'une seule fois.

```
doublon([1, 2, 3, 4, 5])    # False
doublon([1, 2, 3, 2])       # True
doublon([])                 # False
```

Exercice 7 Fréquence d'un élément

Écrire une fonction `frequence(L, x)` qui renvoie le nombre d'occurrences de l'élément `x` dans la liste `L`.

```
frequence([1, 2, 2, 3, 2], 2)    # 3
frequence([5, 5, 5], 5)          # 3
frequence([1, 2, 3], 4)          # 0
```

Exercice 8 Liste incluse

Écrire une fonction `incluse(L1, L2)` qui renvoie `True` si tous les éléments de `L1` sont présents dans `L2`, `False` sinon. L'ordre et les répétitions ne comptent pas : `incluse([1, 1], [1, 2])` renvoie `True`.

```
incluse([1, 2], [1, 2, 3])    # True
incluse([1, 4], [1, 2, 3])    # False
incluse([], [1, 2, 3])        # True
```

Exercice 9 Égalité de listes

Écrire une fonction `egales(L1, L2)` qui renvoie `True` si les deux listes contiennent exactement les mêmes éléments, l'ordre n'ayant pas d'importance, `False` sinon. Autrement dit, les deux listes ont la même taille et chaque valeur apparaît le même nombre de fois dans l'une et dans l'autre. On pourra utiliser la fonction `frequence`.

```
egales([1, 2, 3], [3, 2, 1])    # True
egales([1, 2], [1, 2, 3])       # False
egales([], [])                  # True
egales([1, 1, 2], [1, 2, 2])    # False
```

Exercice 10 Liste croissante

Écrire une fonction `est_croissante(L)` qui renvoie `True` si la liste est strictement croissante (chaque élément est strictement plus grand que le précédent), `False` sinon.

```
est_croissante([1, 2, 3, 4])    # True
est_croissante([1, 2, 2, 3])    # False
est_croissante([5])             # True
```

Exercice 11 Fusion de listes triées

Écrire une fonction `fusion(L1, L2)` qui prend deux listes triées dans l'ordre croissant et renvoie une *nouvelle* liste triée contenant tous les éléments des deux listes. On ne triera pas la liste obtenue : on parcourra les deux listes simultanément en prenant à chaque étape le plus petit des deux éléments courants.

```
fusion([1, 3, 5], [2, 4, 6])    # [1, 2, 3, 4, 5, 6]
fusion([], [1, 2])              # [1, 2]
fusion([1, 2], [])              # [1, 2]
```

Feuille de TD n° 4 – Corrigé

Bases de la programmation – Les listes de listes

Remarque générale : les fonctions s'enchaînent d'un exercice à l'autre (les matrices d'exemple `M`, `A`, `B`… définies dans un exercice sont réutilisées dans les suivants) ; l'ensemble forme un seul programme. Le résultat attendu de chaque appel figure en commentaire. D'autres solutions correctes sont bien sûr possibles.

Exercice 1 Matrice nulle

```
def zeros(n, m):
    M = []                      # la matrice : une liste de lignes
    for i in range(n):
        ligne = []              # une nouvelle ligne, vide
        for j in range(m):
            ligne.append(0)
        M.append(ligne)
    return M

print(zeros(2, 3))              # [[0, 0, 0], [0, 0, 0]]
```

Remarque : Chaque ligne est une *nouvelle* liste créée dans la boucle. L'écriture `[[0] * m] * n` est à proscrire : elle crée `n` références vers la *même* liste, et modifier un élément d'une ligne modifierait toutes les lignes.

Exercice 2 Matrice aléatoire

```
from random import randint

def aleatoire(n, m, a, b):
    M = []
    for i in range(n):
        ligne = []
        for j in range(m):
            ligne.append(randint(a, b))     # entier au hasard entre a et b inclus
        M.append(ligne)
    return M

M = aleatoire(3, 4, 0, 9)
print(M)      # par exemple [[3, 7, 0, 9], [1, 1, 5, 2], [8, 4, 4, 6]]
```

Exercice 3 Affichage

```
def afficher(M, n, m):
    for i in range(n):
        for j in range(m):
            print(M[i][j], end=" ")
        print()                 # fin de la ligne

afficher(M, 3, 4)               # 3 7 0 9
                                # 1 1 5 2
                                # 8 4 4 6
```

Remarque : `print(..., end=" ")` affiche une valeur suivie d'un espace sans passer à la ligne ; le `print()` de la boucle externe termine chaque ligne.

Exercice 4 Occurrences d'une valeur

```
def occurrences(M, n, m, v):
    nb = 0
    for i in range(n):
        for j in range(m):
            if M[i][j] == v:
                nb = nb + 1
    return nb

A = [[1, 2, 3], [4, 1, 6], [1, 8, 9]]
print(occurrences(A, 3, 3, 1))  # 3
```

Exercice 5 Deux lignes identiques

```
def lignes_identiques(M, n, m, i, j):
    for k in range(m):          # on parcourt les colonnes
        if M[i][k] != M[j][k]:
            return False        # une différence suffit
    return True

B = [[1, 2, 3], [4, 5, 6], [1, 2, 3]]
print(lignes_identiques(B, 3, 3, 0, 2))     # True
print(lignes_identiques(B, 3, 3, 0, 1))     # False
```

Remarque : Le `return False` dans la boucle interrompt la comparaison dès la première différence ; si la boucle se termine sans différence, les lignes sont identiques. En Python, `M[i] == M[j]` donnerait le même résultat, mais la méthode élément par élément est celle qui se transpose aux colonnes.

Exercice 6 Existence de deux lignes identiques

```
def deux_lignes_identiques(M, n, m):
    for i in range(n):
        for j in range(i + 1, n):           # chaque paire (i, j) avec i < j
            if lignes_identiques(M, n, m, i, j):
                return True
    return False

print(deux_lignes_identiques(B, 3, 3))      # True
print(deux_lignes_identiques(A, 3, 3))      # False
```

Remarque : La boucle interne commence à `i + 1` : on examine chaque paire de lignes une seule fois et jamais une ligne avec elle-même (deux lignes *distinctes*).

Exercice 7 Colonnes identiques

```
def colonnes_identiques(M, n, m, j, k):
    for i in range(n):          # on parcourt les lignes
        if M[i][j] != M[i][k]:
            return False
    return True

def deux_colonnes_identiques(M, n, m):
    for j in range(m):
        for k in range(j + 1, m):
            if colonnes_identiques(M, n, m, j, k):
                return True
    return False

C = [[1, 2, 1], [3, 4, 3]]
print(deux_colonnes_identiques(C, 2, 3))    # True
print(deux_colonnes_identiques(A, 3, 3))    # False
```

Remarque : Il n'existe pas de liste « colonne » : comparer les colonnes `j` et `k` revient à comparer `M[i][j]` et `M[i][k]` pour chaque ligne `i`. Le reste est identique aux exercices 5 et 6, lignes et colonnes échangées.

Exercice 8 Produit de deux matrices

```
def produit(A, n, m, B, p):
    # A : n lignes, m colonnes ; B : m lignes, p colonnes ; C : n lignes, p colonnes
    C = zeros(n, p)
    for i in range(n):
        for j in range(p):
            s = 0
            for k in range(m):
                s = s + A[i][k] * B[k][j]
            C[i][j] = s
    return C, n, p

A = [[1, 2], [3, 4]]
B = [[5, 6, 7], [8, 9, 10]]
C, n, p = produit(A, 2, 2, B, 3)
afficher(C, n, p)               # 21 24 27
                                # 47 54 61
```

Remarque : Trois boucles imbriquées : `i` et `j` parcourent les cases de `C`, `k` calcule la somme des produits `A[i][k] * B[k][j]`. La fonction renvoie trois valeurs, récupérées par `C, n, p = produit(...)`.

Exercice 9 Matrice symétrique

```
def symetrique(M, n, m):
    if n != m:                  # une matrice symétrique est carrée
        return False
    for i in range(n):
        for j in range(n):
            if M[i][j] != M[j][i]:
                return False
    return True

S = [[1, 2, 3], [2, 5, 6], [3, 6, 9]]
print(symetrique(S, 3, 3))      # True
print(symetrique(B, 2, 3))      # False
```

Remarque : Le test $n \neq m$ évite un dépassement d'indice sur une matrice non carrée. Il suffirait de parcourir les cases au-dessus de la diagonale (`for j in range(i + 1, n)`) puisque chaque paire est vérifiée deux fois.

Exercice 10 Liste et matrice triées

```python
def liste_triee(L):
    for i in range(len(L) - 1):
        if L[i] > L[i + 1]:
            return False
    return True

def matrice_triee(M, n, m):
    for i in range(n):
        if not liste_triee(M[i]):   # M[i] est la ligne d'indice i
            return False
    return True

print(liste_triee([1, 3, 3, 7]))                # True
print(liste_triee([1, 5, 2]))                   # False
print(matrice_triee([[1, 2], [3, 3]], 2, 2))    # True
```

Remarque : `range(len(L) - 1)` s'arrête à l'avant-dernier indice pour que `L[i + 1]` existe toujours. Une liste vide ou à un seul élément est triée : la boucle ne fait aucun tour.

Exercice 11 Tri d'une liste

```python
def passe(L, n):
    aucun_echange = True
    for i in range(n - 1):
        if L[i] > L[i + 1]:
            L[i], L[i + 1] = L[i + 1], L[i]     # échange
            aucun_echange = False
    return aucun_echange

def tri(L, n):
    fini = passe(L, n)
    while not fini:             # tant qu'une passe a fait au moins un échange
        fini = passe(L, n)

L = [5, 1, 4, 2]
print(passe(L, 4), L)           # False [1, 4, 2, 5]
tri(L, 4)
print(L)                        # [1, 2, 4, 5]
```

Remarque : C'est le tri à bulles : à chaque passe, le plus grand élément non encore placé « remonte » en fin de liste. `passe` et `tri` modifient la liste sur place et ne renvoient pas de nouvelle liste : après `tri(L, 4)`, c'est `L` elle-même qui est triée. On pourrait réduire `n` de 1 à chaque passe, le dernier élément étant déjà à sa place.

Exercice 12 Tri d'une matrice

```
def trier_matrice(M, n, m):
    for i in range(n):
        tri(M[i], m)            # trie la ligne d'indice i (modifiée sur place)

M = [[3, 1, 2], [9, 7, 8]]
trier_matrice(M, 2, 3)
print(M)                        # [[1, 2, 3], [7, 8, 9]]
print(matrice_triee(M, 2, 3))   # True
```

Remarque : `M[i]` est la liste de la ligne `i` : la passer à `tri` la trie sur place, donc la matrice est triée sans qu'il soit nécessaire de la renvoyer.

Feuille de TD n° 5

Bases de la programmation – Révision épreuve écrite

Objectif : ce TD vise à tester la compréhension des structures de contrôle, des fonctions, des listes et de la logique algorithmique en Python, dans les conditions de l'épreuve écrite.
Consignes : les seules opérations autorisées sur les listes sont la création d'une liste vide `[]`, `len(L)`, l'accès `L[i]`, `L.append(x)` et le parcours `for x in L`. On n'utilisera ni l'opérateur `in`, ni les méthodes `sort`, `count` et `index`, ni les fonctions `sorted`, `min`, `max` et `sum` : l'objectif est d'écrire soi-même les boucles. Les fonctions d'un exercice peuvent être réutilisées dans les exercices suivants.

Exercice 1 Analyse de programme

Voici un extrait de code :

```
for x in range(1, 10):
    print(x, ": ", end="")
    if x % 2 == 0:
        print("pair")
    elif x % 3 == 0:
        print("triple")
    else:
        print("impair")
```

Expliquer brièvement ce que fait ce programme et indiquer les messages qu'il affiche. Que se passe-t-il pour $x = 6$? Pourquoi ?

Exercice 2 Nombre abondant

Un nombre entier $n \geq 1$ est dit *abondant* si la somme de ses diviseurs propres (ses diviseurs strictement inférieurs à lui-même, 1 compris) est strictement supérieure à lui-même.

Écrire une fonction `abondant(n)` qui renvoie `True` si `n` est abondant, `False` sinon.

```
abondant(12)    # True, car 1 + 2 + 3 + 4 + 6 = 16 > 12
abondant(28)    # False, car 1 + 2 + 4 + 7 + 14 = 28
```

Exercice 3 Liste des nombres abondants

Écrire une fonction `abondants_liste(L)` qui renvoie une nouvelle liste contenant uniquement les nombres abondants de la liste `L`, dans le même ordre.

```
abondants_liste([10, 12, 18, 20])    # [12, 18, 20]
abondants_liste([6, 28, 496])        # []
```

Exercice 4 Position du minimum

Écrire une fonction `position_min(L)` qui renvoie l'indice du plus petit élément de la liste `L`. On suppose que la liste contient au moins un élément ; si le minimum apparaît plusieurs fois, la fonction renvoie l'indice de sa première occurrence.

```
position_min([4, 2, 7, 1, 3])    # 3
position_min([10, 5, 5, 8])      # 1
position_min([42])               # 0
```

Exercice 5 Tri par sélection

Écrire une fonction `tri_selection(L)` qui trie une liste d'entiers `L` dans l'ordre croissant en utilisant l'algorithme du tri par sélection, puis renvoie la liste triée. Rappel de l'algorithme : pour chaque position `i` de 0 à $n-2$, on cherche la position du plus petit élément parmi `L[i]`, …, `L[n - 1]` et on l'échange avec `L[i]`.

```
tri_selection([5, 2, 9, 1])    # [1, 2, 5, 9]
tri_selection([3, 3, 2])       # [2, 3, 3]
```

Exercice 6 Doublons optimisés

Écrire une fonction `doublon(L)` qui renvoie `True` si la liste contient au moins un doublon (une valeur présente au moins deux fois), `False` sinon. La fonction ne doit pas faire plus de $n(n-1)/2$ comparaisons, `n` étant la taille de la liste : chaque paire d'éléments n'est comparée qu'une seule fois.

```
doublon([1, 2, 3, 4, 5])    # False
doublon([1, 2, 3, 2])       # True
doublon([])                 # False
```

Exercice 7 Fréquence d'un élément

Écrire une fonction `frequence(L, x)` qui renvoie le nombre d'occurrences de l'élément `x` dans la liste `L`.

```
frequence([1, 2, 2, 3, 2], 2)    # 3
frequence([5, 5, 5], 5)          # 3
frequence([1, 2, 3], 4)          # 0
```

Exercice 8 Liste incluse

Écrire une fonction `incluse(L1, L2)` qui renvoie `True` si tous les éléments de `L1` sont présents dans `L2`, `False` sinon. L'ordre et les répétitions ne comptent pas : `incluse([1, 1], [1, 2])` renvoie `True`.

```
incluse([1, 2], [1, 2, 3])    # True
incluse([1, 4], [1, 2, 3])    # False
incluse([], [1, 2, 3])        # True
```

Exercice 9 Égalité de listes

Écrire une fonction `egales(L1, L2)` qui renvoie `True` si les deux listes contiennent exactement les mêmes éléments, l'ordre n'ayant pas d'importance, `False` sinon. Autrement dit, les deux listes ont la même taille et chaque valeur apparaît le même nombre de fois dans l'une et dans l'autre. On pourra utiliser la fonction `frequence`.

```
egales([1, 2, 3], [3, 2, 1])    # True
egales([1, 2], [1, 2, 3])       # False
egales([], [])                  # True
egales([1, 1, 2], [1, 2, 2])    # False
```

Exercice 10 Liste croissante

Écrire une fonction `est_croissante(L)` qui renvoie `True` si la liste est strictement croissante (chaque élément est strictement plus grand que le précédent), `False` sinon.

```
est_croissante([1, 2, 3, 4])    # True
est_croissante([1, 2, 2, 3])    # False
est_croissante([5])             # True
```

Exercice 11 Fusion de listes triées

Écrire une fonction `fusion(L1, L2)` qui prend deux listes triées dans l'ordre croissant et renvoie une *nouvelle* liste triée contenant tous les éléments des deux listes. On ne triera pas la liste obtenue : on parcourra les deux listes simultanément en prenant à chaque étape le plus petit des deux éléments courants.

```
fusion([1, 3, 5], [2, 4, 6])    # [1, 2, 3, 4, 5, 6]
fusion([], [1, 2])              # [1, 2]
fusion([1, 2], [])              # [1, 2]
```

Feuille de TD n° 5 – Corrigé

Bases de la programmation – Révision épreuve écrite

Remarque générale : les solutions respectent les consignes du sujet (aucune méthode de liste autre que `append`, pas d'opérateur `in`). Les fonctions s'enchaînent d'un exercice à l'autre : `abondant` sert à l'exercice 3, `frequence` aux exercices 8 et 9. Le résultat attendu de chaque appel figure en commentaire.

Exercice 1 Analyse de programme

Le programme parcourt les entiers `x` de 1 à 9 (`range(1, 10)` exclut 10). Pour chacun, il affiche `x` suivi de « : » sans passer à la ligne, puis un mot choisi par les tests, *dans l'ordre* : « pair » si `x` est divisible par 2, sinon « triple » si `x` est divisible par 3, sinon « impair ». Affichage obtenu :

```
1 : impair
2 : pair
3 : triple
4 : pair
5 : impair
6 : pair
7 : impair
8 : pair
9 : triple
```

Pour $x = 6$, les deux conditions sont vraies, mais un bloc `if` / `elif` / `else` n'exécute que la première branche dont la condition est vraie : 6 est affiché « pair », jamais « triple ». Seuls 3 et 9 (impairs et multiples de 3) affichent « triple ».

Exercice 2 Nombre abondant

```python
def abondant(n):
    somme = 0
    for d in range(1, n):       # diviseurs propres : de 1 à n - 1
        if n % d == 0:
            somme = somme + d
    return somme > n

print(abondant(12))             # True
print(abondant(28))             # False
```

Remarque : `range(1, n)` parcourt les candidats diviseurs de 1 à $n - 1$: n lui-même n'est pas un diviseur propre. La fonction renvoie directement le booléen `somme > n`. Pour $n = 1$, la somme vaut 0 et la fonction renvoie `False`.

Exercice 3 Liste des nombres abondants

```
def abondants_liste(L):
    resultat = []
    for x in L:
        if abondant(x):
            resultat.append(x)
    return resultat

print(abondants_liste([10, 12, 18, 20]))    # [12, 18, 20]
print(abondants_liste([6, 28, 496]))        # []
```

Remarque : Schéma classique de *filtrage* : on part d'une liste vide et on y ajoute les éléments qui vérifient la propriété, dans l'ordre du parcours.

Exercice 4 Position du minimum

```
def position_min(L):
    pos = 0                     # indice du plus petit élément vu jusqu'ici
    for i in range(1, len(L)):
        if L[i] < L[pos]:       # strictement : on garde la première occurrence
            pos = i
    return pos

print(position_min([4, 2, 7, 1, 3]))        # 3
print(position_min([10, 5, 5, 8]))          # 1
print(position_min([42]))                   # 0
```

Remarque : La comparaison stricte `L[i] < L[pos]` garantit que, pour des minimums égaux, c'est le premier rencontré qui est conservé (`[10, 5, 5, 8]` donne 1). Le parcours commence à l'indice 1 puisque l'indice 0 sert d'initialisation.

Exercice 5 Tri par sélection

```
def tri_selection(L):
    n = len(L)
    for i in range(n - 1):
        # position du plus petit élément parmi L[i], ..., L[n - 1]
        pos = i
        for j in range(i + 1, n):
            if L[j] < L[pos]:
                pos = j
        L[i], L[pos] = L[pos], L[i]         # on le place en position i
    return L

print(tri_selection([5, 2, 9, 1]))          # [1, 2, 5, 9]
print(tri_selection([3, 3, 2]))             # [2, 3, 3]
```

Remarque : La boucle interne est la recherche du minimum de l'exercice 4, restreinte aux indices `i` à $n-1$; l'échange place ce minimum en position `i`. Après l'étape `i`, les `i + 1` premiers éléments sont triés et définitivement placés. La liste est triée sur place, puis renvoyée pour respecter les exemples de l'énoncé.

Exercice 6 Doublons optimisés

```
def doublon(L):
    n = len(L)
    for i in range(n):
        for j in range(i + 1, n):           # chaque paire comparée une seule fois
            if L[i] == L[j]:
                return True
    return False

print(doublon([1, 2, 3, 4, 5]))             # False
print(doublon([1, 2, 3, 2]))                # True
print(doublon([]))                          # False
```

Remarque : La boucle interne commence à `i + 1` : la paire (i, j) n'est examinée qu'une fois et jamais un élément avec lui-même, soit $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$ comparaisons au maximum. Le `return True` arrête tout dès le premier doublon.

Exercice 7 Fréquence d'un élément

```
def frequence(L, x):
    nb = 0
    for e in L:
        if e == x:
            nb = nb + 1
    return nb

print(frequence([1, 2, 2, 3, 2], 2))        # 3
print(frequence([5, 5, 5], 5))              # 3
print(frequence([1, 2, 3], 4))              # 0
```

Exercice 8 Liste incluse

```
def incluse(L1, L2):
    for x in L1:
        if frequence(L2, x) == 0:           # x n'est pas dans L2
            return False
    return True

print(incluse([1, 2], [1, 2, 3]))           # True
print(incluse([1, 4], [1, 2, 3]))           # False
print(incluse([], [1, 2, 3]))               # True
```

Remarque : « `x` est dans `L2` » s'écrit `frequence(L2, x) > 0` sans l'opérateur `in`. Pour une liste `L1` vide, la boucle ne fait aucun tour et la fonction renvoie `True`.

Exercice 9 Égalité de listes

```python
def egales(L1, L2):
    if len(L1) != len(L2):
        return False
    for x in L1:
        if frequence(L1, x) != frequence(L2, x):
            return False
    return True

print(egales([1, 2, 3], [3, 2, 1]))         # True
print(egales([1, 2], [1, 2, 3]))            # False
print(egales([], []))                       # True
print(egales([1, 1, 2], [1, 2, 2]))         # False
```

Remarque : Deux tests `incluse(L1, L2) and incluse(L2, L1)` ne suffiraient pas : `[1, 1, 2]` et `[1, 2, 2]` seraient déclarées égales. Comparer les fréquences de chaque valeur de `L1`, à tailles égales, garantit que les deux listes contiennent exactement les mêmes éléments avec les mêmes répétitions.

Exercice 10 Liste croissante

```python
def est_croissante(L):
    for i in range(len(L) - 1):
        if L[i] >= L[i + 1]:
            return False
    return True

print(est_croissante([1, 2, 3, 4]))         # True
print(est_croissante([1, 2, 2, 3]))         # False
print(est_croissante([5]))                  # True
```

Remarque : Le test `L[i] >= L[i + 1]` est la négation de « strictement croissante » : une égalité entre deux éléments consécutifs suffit à renvoyer `False`. Une liste vide ou à un seul élément est croissante (aucun tour de boucle).

Exercice 11 Fusion de listes triées

```
def fusion(L1, L2):
    resultat = []
    i = 0                       # position courante dans L1
    j = 0                       # position courante dans L2
    while i < len(L1) and j < len(L2):
        if L1[i] <= L2[j]:      # on prend le plus petit des deux
            resultat.append(L1[i])
            i = i + 1
        else:
            resultat.append(L2[j])
            j = j + 1
    while i < len(L1):          # il reste des éléments dans L1
        resultat.append(L1[i])
        i = i + 1
    while j < len(L2):          # il reste des éléments dans L2
        resultat.append(L2[j])
        j = j + 1
    return resultat

print(fusion([1, 3, 5], [2, 4, 6]))         # [1, 2, 3, 4, 5, 6]
print(fusion([], [1, 2]))                   # [1, 2]
print(fusion([1, 2], []))                   # [1, 2]
```

Remarque : Deux indices avancent chacun dans leur liste ; à chaque étape on copie le plus petit des deux éléments courants, ce qui produit une liste triée en un seul parcours. Quand une liste est épuisée, il reste à recopier la fin de l'autre : c'est le rôle des deux dernières boucles (une seule des deux fait effectivement des tours). Le `<=` conserve les éléments égaux des deux listes.